

Se alguém tiver um link mais rápido, seria aconselhável dar uma olhada em <http://www.acm.org/contest/96/finals/problems.html>, pois ela tem algumas figuras. De qualquer forma, os problemas abaixo contêm a parte de texto da página acima.

Problema A

10-20-30

Arquivo de entrada: 10-20-30.in

Um jogo de cartas simples de paciência chamado 10-20-30 usa um baralho padrão de 52 cartas de baralho em que o naipe é irrelevante. O valor de uma carta de face (rei, rainha, valete) é 10. O valor de um ás é um. O valor de cada uma das outras cartas é o valor de face da carta (2, 3, 4, etc.). As cartas são distribuídas do topo do baralho. Você começa distribuindo sete cartas, da esquerda para a direita, formando sete pilhas. Depois de jogar uma carta na pilha mais à direita, a próxima pilha na qual você joga uma carta é a pilha mais à esquerda.

Para cada carta colocada em uma pilha, verifique essa pilha para ver se uma das As três combinações de cartas seguintes totalizam 10, 20 ou 30.

1. os dois primeiros e o último,
2. o primeiro e os dois últimos, ou
3. as três últimas cartas.

Se for o caso, pegue as três cartas e coloque-as na parte inferior da mesa. baralho. Para esse problema, sempre verifique a pilha na ordem em que se encontra descrito. Recolha as cartas na ordem em que elas aparecem na pilha e coloque-as no fundo do baralho. Pegar três cartas pode expor mais três cartas que podem ser retiradas. Se for o caso, pegue-as. Continue até que não haja mais conjuntos de três na pilha.

Por exemplo, suponha que uma pilha contenha 5 9 7 3, sendo que o 5 está no primeira carta do monte e, em seguida, um 6 é jogado. As duas primeiras cartas mais a última carta (5 + 9 + 6) somam 20. O novo conteúdo da pilha depois de pegar essas três cartas se torna 7 3. Além disso, a carta mais baixa A carta do baralho agora é o 6, a carta acima dele é o 9 e a carta acima do 9 está o 5.

Se uma rainha fosse jogada em vez do seis, 5 + 9 + 10 = 24, e 5 + 3 + 10 = 18, mas 7 + 3 + 10 = 20, então as últimas três cartas seriam recolhidos, deixando a pilha com 5 9.

Se uma pilha contiver apenas três cartas, quando as três somarem 10, 20 ou 30, então a pilha "desaparece" quando as cartas são recolhidas. Ou seja, A jogada subsequente pula a posição em que a pilha agora vazia ocupada. Você ganha se todas as pilhas desaparecerem. Você perde se estiver incapaz de distribuir uma carta. Também é possível haver um empate se nenhum dos dois das duas condições anteriores jamais ocorrer.

Escreva um programa que jogará jogos de 10-20-30 com uma carta inicial decks como entrada.

Entrada

Cada conjunto de entrada consiste em uma sequência de 52 números inteiros separados por espaços e/ou finais de linha. Os números inteiros representam valores de cartão do baralho inicial para esse jogo. O primeiro número inteiro é a carta do topo do baralho deck. A entrada é encerrada por um único zero (0) após o último convés.

Saída

Para cada conjunto de entrada, imprima se o resultado do jogo é uma vitória, uma derrota ou um empate, e imprima o número de vezes que uma carta é distribuída antes de

os resultados do jogo podem ser determinados. (Um empate ocorre assim que o estado do jogo é repetido.) Use o formato mostrado na seção "Saída para a entrada de amostra".

Entrada de amostra

```
2 6 5 10 10 4 10 10 10 4 5 10 4 5 10 9 7 6 1 7 6 9 5 3 10 10 4 10 9 2 1
10 1 10 10 10 3 10 9 8 10 8 7 1 2 8 6 7 3 3 8 2
4 3 2 10 8 10 6 8 9 5 8 10 5 3 5 4 6 9 9 1 7 6 3 5 10 10 8 10 9 10 10 7
2 6 10 10 4 10 1 3 10 1 1 10 2 2 10 4 10 7 7 10
10 5 4 3 5 7 10 8 2 3 9 10 8 4 5 1 7 6 7 2 6 9 10 2 3 10 3 4 4 9 10 1 1
10 5 10 10 1 8 10 7 8 10 6 10 10 9 6 2 10 10
0
```

Saída para a entrada de amostra

Vitória : 66
Perda: 82
Empate: 73

Problema B

Círculos de chamada

Arquivo de entrada: circles.in

Se você tem assistido a comerciais de televisão de empresas de telefonia de longa distância ultimamente, percebeu que muitas empresas têm g a s t a d o muito dinheiro tentando convencer as pessoas de que oferecem o melhor serviço pelo menor custo. Uma empresa tem "círculos de chamadas". Você fornece uma lista de pessoas para as quais liga com mais frequência. Se você ligar para alguém do seu círculo de chamadas (que também seja cliente da mesma empresa), terá descontos maiores do que se ligar para alguém de fora do seu círculo. Outra empresa ressaltava que você só obtém os grandes descontos para as pessoas do seu círculo de chamadas e, se você mudar a pessoa para quem liga com mais frequência, cabe a você adicioná-la ao seu círculo de chamadas.

Por exemplo, se Ben chamasse Dolly, isso seria representado no arquivo de dados como

Ben Dolly

A entrada é encerrada por valores de zero (0) para n e m.

Saída

Para cada conjunto de entrada, imprima uma linha de cabeçalho com o número do conjunto de dados, seguido de uma linha para cada círculo de chamada nesse conjunto de dados. Cada A linha do círculo de chamada contém os nomes de todas as pessoas em qualquer ordem dentro do círculo, separados por vírgula-espaco (uma vírgula seguida de um espaço). Os conjuntos de saída são separados por linhas em branco.

Entrada de amostra

```
5 6
Ben Alexander
Alexander Dolly
Dolly Ben
Dolly Benedict
Benedict Dolly
Alexander Aaron
14 34
John Aaron
Aaron Benedict
Betsy John
Betsy Ringo
Ringo Dolly
Bento Paulo
John Betsy
John Aaron
Benedict George
Dolly Ringo
Paul Martha
George Ben
Alexander George
Betsy Ringo
Alexander Stephen
Martha Stephen
Benedict Alexander
Stephen Paul
Betsy Ringo
Quincy Martha
Ben Patrick
Betsy Ringo
```

```
Patrick Stephen
Paul Alexander
Patrick Ben
Stephen Quincy
Ringo Betsy
Betsy Benedict
Betsy Benedict
Betsy Benedict
```

Betsy Benedict

Betsy Benedict

Betsy Benedict
Quincy Martha
0 0

Saída para a entrada de amostra

Círculos de chamada para o conjunto de dados 1:

Ben, Alexander, Dolly, Benedict
Aaron

Círculos de chamada para o conjunto de dados 2:

John, Betsy, Ringo e Dolly
Aaron
Benedito
Paul, George, Martha, Ben, Alexander, Stephen, Quincy, Patrick

Problema C

Cortando cantos

Arquivo de entrada: corner.in

Mensageiros de bicicleta que entregam documentos e pequenos itens a empresas há muito tempo fazem parte dos serviços de transporte de guerrilha em várias das principais cidades dos EUA. Os ciclistas de Boston são uma raça rara de ciclistas. Eles são famosos por sua velocidade, seu desrespeito pela ruas de mão única e sinais de trânsito, e seu desprezo descarado por carros, táxis, ônibus e pedestres.

Os serviços de mensageiro de bicicleta são muito competitivos. O Billy's Bicycle Messenger Service não é exceção. Para aumentar sua vantagem competitiva e determinar suas despesas reais, a BBMS está desenvolvendo um novo esquema de preços de entregas que depende da rota mais curta que os mensageiros podem percorrer. Você deve escrever um programa para ajudar a BBMS a determinar as distâncias para essas rotas.

As suposições a seguir ajudam a simplificar sua tarefa:

Os mensageiros podem andar de bicicleta em qualquer lugar no nível do solo, exceto dentro dos edifícios. Os andares térreos de edifícios com formas irregulares são modelados pela união dos interiores de retângulos. Por convenção, todos os retângulos que se cruzam compartilham o espaço interior e fazem parte do mesmo edifício. Os

A LibertyBell Phone Co. é uma nova empresa que acha que tem o plano de chamadas que pode tirar as outras empresas do mercado. A LibertyBell tem círculos de chamadas, mas eles calculam o seu círculo de chamadas para você. É assim que funciona. A LibertyBell mantém o controle de todas as chamadas telefônicas. Além de você, seu círculo de chamadas consiste em todas as pessoas para quem você liga e que ligam para você, direta ou indiretamente. Por exemplo, se Ben liga para Alexander, Alexander liga para Dolly e Dolly liga para Ben, todos eles estão dentro do mesmo círculo. Se Dolly também liga para Benedict e Benedict liga para Dolly, então Benedict está no mesmo círculo de chamadas que Dolly, Ben e Alexander. Por fim, se Alexandre chama Arão, mas Arão não chama Alexandre, Ben, Dolly ou Bento, então Arão não está no círculo. Você foi contratado pela LibertyBell para escrever o programa para determinar o círculo de chamada Entrada O arquivo de entrada conterá um ou mais conjuntos de dados. Cada conjunto de dados começa com uma linha que contém dois números inteiros, n e m. O primeiro número inteiro, n, representa o número de pessoas diferentes que estão no conjunto de dados. O valor máximo para n é 25. O restante do conjunto de dados consiste em m linhas, cada uma representando uma chamada telefônica. Cada chamada é representada por dois nomes, separados por um espaço simples. Os nomes são apenas nomes próprios (exclusivos em um conjunto de dados), diferenciam maiúsculas de minúsculas e consistem apenas em caracteres alfabéticos; nenhum nome tem mais de 25 letras. Para	retângulos definidores de dois edifícios separados nunca se tocam, embora possam estar bem próximos. (Mensageiros de bicicleta - magros até demais - podem se deslocar entre dois prédios quaisquer. Eles podem cortar os cantos mais afiados e passar com seus pneus magros pelos perímetros dos edifícios). Os pontos de partida e parada nunca estão dentro dos edifícios. Sempre há uma rota do ponto de partida até o ponto de parada. Seu programa deve ser capaz de processar vários cenários. Cada cenário define os edifícios e os pontos de partida e parada de uma rota de entrega. A figura abaixo mostra uma visão panorâmica de um cenário típico. O arquivo de entrada representa vários cenários. A entrada para cada cenário consiste em linhas como as seguintes: Primeira linha: n O número de retângulos que descrevem os edifícios no cenário. 0 <= n <= 20 Segunda linha: x1 y1 x2 y2 Os valores de x e coordenadas y dos pontos de partida e parada da rota. Restantes n linhas: x1 y1 x2 y2 x3 y3 As coordenadas x e y de três vértices de um retângulo. As coordenadas x e y de todos os dados de entrada são números reais entre 0 e 1000, inclusive. As coordenadas sucessivas em uma linha são separadas por um ou mais espaços em branco. O número inteiro -1 segue os dados do último cenário. O resultado deve numerar cada cenário (Cenário 1, Cenário 2, etc.) e fornecer a distância da rota mais curta do ponto de partida ao ponto de parada, conforme ilustrado no exemplo de resultado abaixo. A distância deve
deve ser escrito com dois dígitos à direita do ponto decimal. A saída para cenários sucessivos deve ser separada por linhas em branco. Entrada de amostra <pre>5 6.5 9 10 3 1 5 3 3 6 6 5.25 2 8 2 8 3.5 6 10 6 12 9 12 7 6 11 6 11 8 10 7 11 7 11 11 -1</pre> Saída de amostra Cenário n° 1 Distância da rota: 7,28 Problema D Toque o tambor lentamente Arquivo de entrada: drum.in Há muitos anos, a "memória primária" da maioria dos sistemas de computador era um tambor magnético. Os cabeçotes de leitura/gravação eram posicionados de forma que pudessem acessar os dados da superfície externa magnética à medida que o tambor girava ao longo de seu eixo horizontal. A ilustração a seguir apresenta a ideia básica: À medida que o tambor girava, a palavra de dados sob o(s) cabeçote(s) de leitura/gravação podia ser acessada. O tambor continuava a girar depois que uma instrução era obtida. Após a execução de uma instrução, a palavra pronta para ser acessada pelo(s) cabeçote(s) de leitura/gravação normalmente estava a muitas palavras de distância. Para minimizar o atraso que ocorreria se as instruções a serem executadas sequencialmente fossem colocadas em palavras consecutivas no tambor, os projetistas dessas máquinas frequentemente incluíam o endereço do tambor da próxima instrução como um campo na instrução (ou seja, cada instrução incluía um endereço explícito de "próxima instrução"). Assim, os montadores "otimizadores" podiam preencher o campo da próxima instrução com o endereço da primeira palavra disponível pronta para ser lida pelo tambor assim que a instrução atual fosse concluída. Neste problema, queremos determinar os tempos médios de execução de programas simples sem loops. Consideraremos apenas um único cabeçote de leitura/gravação em uma única trilha. Suponha que as palavras nessa trilha tenham endereços sequenciais numerados de 1 a n. Todas as instruções requerem o mesmo tempo para serem executadas, especificamente o mesmo tempo que o tambor leva para girar passando por t palavras. t não inclui o tempo para ler a instrução do tambor, nem inclui o atraso adicional de rotação que pode ser necessário se a próxima instrução não estiver no endereço "ideal". Entretanto, esses fatores devem ser incluídos no cálculo do tempo médio de execução. Há três tipos de instruções: terminal, condicional e incondicional. As instruções terminais não têm um endereço de "próxima instrução", pois encerram a execução de um programa. As instruções condicionais têm dois endereços de "próxima instrução", e as instruções incondicionais têm apenas um endereço de "próxima instrução". O tempo de execução de um programa é o tempo gasto desde o início da leitura da primeira instrução até a execução da instrução terminal. Para calcular o tempo médio de execução de um programa, cada tempo de execução possível é ponderado (multiplicado) pela probabilidade de o	Caso 1. Tempo de execução = 6,0000 Caso 2. Tempo de execução = 21,0000 Caso 3. Tempo de execução = 12,0000 Caso 4. Tempo de execução = 12,5000 Caso 5. Tempo de execução = 26,5000 Problema E Pattern Matching Prelims Arquivo de entrada: pattern.in Alguns algoritmos de reconhecimento óptico de caracteres comparam uma imagem digitalizada com modelos de caracteres "perfeitos". Parte da dificuldade com essas comparações é decidir por onde começar a comparação. Isso ocorre porque os caracteres na imagem digitalizada estão sujeitos a ruído e distorção, resultando em alterações de tamanho, posição e orientação. Um procedimento que às vezes é usado para lidar com mudanças de posição corresponde ao "centro de gravidade" do caractere digitalizado e aos modelos com os quais ele é comparado. Neste problema, você deve determinar os "centros de gravidade" de imagens digitalizadas de caracteres. Para nossos propósitos, uma imagem digitalizada será uma matriz retangular de números reais, cada um dos quais representa o valor da escala de cinza de um ponto em uma imagem digitalizada. Cada valor de escala de cinza estará entre 0 (representando uma região totalmente branca) e 1 (representando uma região totalmente preta). A matriz não terá mais de 25 linhas e 25 colunas. O centro de gravidade é um elemento específico de uma matriz. Suponha que o centro de gravidade esteja na i-ésima linha e na j-ésima coluna. Então, a diferença entre a soma dos elementos das duas partes da matriz acima e abaixo da i-ésima linha é mínima. Da mesma forma, a diferença das somas dos elementos nas duas partes da matriz à esquerda e à direita da j-ésima coluna é mínima. Considere a matriz mostrada abaixo, que pode ter resultado da digitalização de um "o" minúsculo. O centro de gravidade dessa matriz está exclusivamente na linha 3, coluna 3. A diferença da soma dos elementos em cada parte da matriz formada ao ignorar a terceira linha é 0,1 (as somas são 5,55 e 5,65). A diferença da soma de cada parte da matriz formada ao ignorar a terceira coluna é 0,0 (as somas são 5,60). Entrada A entrada consistirá em uma sequência de imagens de caracteres digitalizados. A entrada para cada imagem começará com dois números inteiros especificando o número de linhas e colunas nos dados digitalizados. Isso será imediatamente seguido pelos dados digitalizados em escala de cinza fornecidos em ordem crescente de linha. Um par de zeros seguirá os dados da última imagem de entrada. Saída Para cada imagem de caractere de entrada, exiba seu número (eles são numerados sequencialmente a partir de 1) e a linha e a coluna correspondentes ao centro de gravidade. Se houver mais de um centro de gravidade, deverá ser exibido aquele que tiver a maior linha e coluna. O exemplo a seguir ilustra um formato de saída razoável. Entrada de amostra
correr. Assumimos a mesma probabilidade de tomar cada caminho de um ramo em uma instrução condicional. A soma de todos os tempos de execução ponderados é o tempo médio de execução do programa. Pressupostos No início de cada caso de teste, o tambor é posicionado de forma que o no local 1 está prestes a ser lida. Cada programa começa com a palavra no local 1. O tempo para ler uma instrução é uma unidade de tempo. Sempre haverá pelo menos uma instrução de terminal, mas pode haver várias. Entrada A entrada consiste em um número de casos de teste. A entrada para cada teste O case começa com uma linha que contém valores inteiros para n (1 < n < 50) e t (0 < t < n). Essa linha é seguida por uma sequência de linhas, cada uma com dos quais contém números inteiros que representam um endereço de instrução e zero, um ou dois endereços de ramificação. Especificamente, para cada instrução há um local (entre 1 e n), o número de "próximos endereços de "instrução" (0 para uma instrução de terminal, 1 para uma instrução de instrução incondicional, e 2 para uma instrução condicional), e esse número de endereços de ramificação. A última instrução é seguida por 0 em um linha por si só. O conjunto de entrada é encerrado por valores de 0 para ambos n e t. Saída Para cada caso de teste, imprima o número do caso (eles são numerados sequencialmente, começando com 1) e o tempo médio de execução para o	<pre>5 5 0.1 0.2 0.1 0.2 0.1 0.1 0.2 0.3 0.1 0.1 0.2 0.3 0.1 0.1 0.3 0.4 0.1 0.1 0.1 0.2 0.2 0.2 0.3 0.3 0.1</pre> <pre>5 10 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.2 0.2 0.2 0.2 0.2 0.2 0.2 0.2 0.2 0.2 0.3 0.3 0.3 0.3 0.3 0.3 0.3 0.3 0.3 0.3 0.4 0.4 0.4 0.4 0.4 0.4 0.4 0.4 0.4 0.4 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.6</pre> 0 0 Saída para a entrada de amostra Caso 1: centro em (3, 3) Caso 2: centro em (4, 6) Problema F Autômatos de treliça não determinísticos Arquivo de entrada: trellis.in Um autômato de treliça não determinístico (NTA) é um tipo de autômato paralelo máquina composta por processadores idênticos de estado finito dispostos em um treliça triangular infinita. O topo, ou vértice, do triângulo é um processador único. A próxima linha tem dois processadores e cada A linha de um NTA tem um processador a mais do que a linha acima dela. Cada

programa. Os tempos de execução devem ser precisos e exibidos com quatro dígitos fracionários. Entrada de amostra <pre>10 5 1 0 0 10 5 1 1 6 6 0 0 10 5 1 1 7 7 0 0 10 5 1 2 7 8 7 0 8 0 0 10 6 8 0 7 1 3 3 0 1 2 7 8 0 0 0</pre> Saída para a entrada de amostra	em um NTA está conectado a dois filhos na linha abaixo dele. A computação em um NTA ocorre de baixo para cima; o estado de cada processador em uma linha é baseada no estado dos filhos do processador e um tabela de transição. A entrada para uma NTA é a configuração inicial de uma linha de processadores. A entrada é especificada por uma string que fornece o estado inicial de cada processador em uma linha, de modo que um caractere n especifica a configuração inicial para uma linha de n processadores. A computação prossegue pela NTA até o ápice de forma não determinística calcular o estado de cada processador em uma linha com base no tabela de transição e o estado dos filhos do processador na linha abaixo. Alguns estados são identificados como estados de aceitação. Algumas transições são computados de forma não determinística. Uma entrada é aceita se algum coloca o processador apex em um estado de aceitação. Uma entrada é rejeitado se nenhuma computação colocar o processador de vértice em um estado de aceitação. Por exemplo, a tabela abaixo mostra as transições de um NTA de 3 estados. Os estados são rotulados pelos caracteres "a", "b" e "c"; o o único estado aceito é "c". O diagrama abaixo mostra dois cálculos para a entrada "bba". Os O cálculo à esquerda rejeita a entrada, pois o estado do vértice é "a"; mas o cálculo à direita aceita a entrada, pois o o estado do vértice é "c". Como alguns cálculos resultam em um estado de aceitação para o vértice, a entrada "bba" é aceita pelo NTA. A entrada "bbb" seria rejeitada por esse NTA, pois a única O cálculo resulta no estado "a" para o vértice.
Entrada Os estados (e entradas) de um NTA são letras minúsculas consecutivas. Assim, os estados de uma NTA de 5 estados são "a", "b", "c", "d" e "e". Os estados aceitantes são agrupados no final das cartas para que, se um o NTA de 5 estados tem dois estados de aceitação, os estados de aceitação são "d" e "e". A entrada para seu programa é uma sequência de descrições de NTA e configurações iniciais. Uma descrição de NTA é dada pelo número de estados n seguidos pelo número de estados de aceitação em uma linha separados por espaços em branco. A tabela de transição n x n segue em ordem maior de linha; cada string de transição é fornecida em uma linha separada. Cada descrição de NTA é seguida por uma sequência de configurações, uma por linha. Uma linha de "##" encerra a sequência de configurações iniciais para essa NTA. Uma descrição de NTA na qual o o número de estados for zero, encerra a entrada do seu programa. Os NTAs terão no máximo 15 estados, a configuração inicial será de máximo de 15 caracteres. Saída Para cada descrição de NTA, imprima o número da NTA (NTA 1, NTA 2, etc.). Para cada configuração inicial de um NTA, imprima a palavra "accept" (aceitar) ou "reject" (rejeitar) seguido de uma cópia da configuração inicial. Entrada de amostra <pre>3 1 a a c ca a b c b a bba aaaaa abab babbbba a baaab abbbabá baba bcbab # 3 2 ab a c a ab b c b ab abc cbc # 0 0</pre>	começa com uma linha contendo três números inteiros, c s d, em que c é o número central, 0 <= c <= 99, s é o número de portas de decapagem em centro c, 0 <= s <= 10, e d é o número de portas de relé no centro c, 0 <= d <= 10. Seguem-se então d linhas, uma para cada porta de relé. Cada uma dessas linhas contém três números inteiros, r v l, em que r é o número do centro de retransmissão, 0 <= r <= 99, v é o volume total de remessas para esse centro no dia, expressas como uma porcentagem de volume do reboque, 0 <= v <= 900 e l é o último tempo aceitável para remessas para chegar ao centro r, expresso em minutos desde o início do dia, 0 <= l <= 1440. (v > 100 indica que mais de um único trailer deve ser usado). A segunda parte da entrada descreve parte do tráfego do dia. Esse A parte começa com um número inteiro m em uma linha isolada, indicando o número de registros de chegada de reboque que se seguem, 1 <= m <= 100. Cada o registro começa com uma linha que contém três números inteiros, a c s, em que a é o horário de chegada do trailer expresso em minutos desde o início da dia, 0 <= a <= 1440, no centro de número c, e s é o número de remessas no trailer, 0 <= s <= 10. Então, todas as remessas s são descrito por linhas de 5 números inteiros, i o r v t, representando o código de identificação de remessa i, 0 <= A segunda parte da entrada descreve parte do tráfego do dia. Esta parte começa com um inteiro n em uma linha isolada, indicando o número de registros de chegada do trailer que se seguem, 1 <= m <= 100. Cada registro começa com uma linha contendo três números inteiros, a c s, em que a é o horário de chegada do trailer expresso como minutos desde o início do dia, 0 <= a <= 1440, no centro número c, e s é o número de remessas no trailer, 0 <= s <= 10. Então, todos os s carregamentos são descritos por s linhas de 5 números inteiros, i o r v t, representando o código de identificação de remessa i, 0 <= i <= 99, os números de origem e do próximo centro de retransmissão o e r, respectivamente, o volume da remessa v como uma porcentagem do volume da carreta e o tempo t necessário para viajar do centro c até o destino r, medido em minutos. t é zero se c for igual a r. Os registros de chegada estão em ordem de valores crescentes de a. Não há dois registros com o mesmo par (a,c). Todos os registros Os números centrais usados como valores para c e r terão uma definição correspondente na primeira parte da entrada, embora a Os números centrais usados para o não precisam. Saída Para cada um dos n ICPCs, seu programa deve escrever uma linha descrevendo o tempo médio de espera para a decapagem de portas na unidade apropriada de essas duas formas: A espera média por uma porta de decapagem no ICPC c é de ###.# minutos. Não há espera para uma porta de decapagem no ICPC c. O tempo médio de espera é afetado apenas pelos reboques que esperam pelo menos um minuto para uma porta de remoção. Seu programa deve listar todas as remessas, sendo que qualquer parte delas não chegarem a seus destinos intermediários ou finais por nenhuma das últimos horários de chegada indicados ao longo da rota. Esse relatório deve aparecer em colunas com os cabeçalhos mostrados: As remessas atrasadas são: Id Origem Destino Volume Entrada de amostra: <pre>2 0 1 1 8 40 600 8 3 4</pre>
Saída para a entrada de amostra NTA 1 aceitar bba rejeitar abab aceitar babbbba rejeitar um rejeitar aaaaa aceitar baaab aceitar abbbaba aceitar baba rejeitar bcbab NTA 2 rejeitar abc aceitar cbc Problema G Caminhões Arquivo de entrada: truck.in A Allied Container Movers (ACM) é uma empresa de transporte rodoviário que fornece entrega de frete durante a noite. A ACM possui uma rede de distribuição com vários centros de processamento de contêineres intermediários (ICPCs). Em um ICPC,	<pre>6 115 1200 2 95 1260 10 100 1440 4 55 1380 7 500 0 1 17 11 8 40 80 700 8 3 24 11 8 45 0 18 11 6 40 120 23 11 10 15 600 720 8 1 16 3 8 100 0 750 8 2 4 15 2 50 180 7 15 6 50 120 760 8 4 14 3 4 20 300 27 3 2 20 180 33 3 10 35 600 16 3 6 25 120 780 8 2 12 9 2 25 180 15 9 4 35 300 800 8 1 19 18 10 50 600</pre>

um reboque de entrada é descarregado em uma porta de remoção. Frete destinado para esse centro é simplesmente confirmada como recebida. Remessas subsequentes são distribuídos para portas de retransmissão com base em seus próximos destinos, onde eles são carregados em carretas de espera. Cada ICPC tem várias portas de remoção para descarregar os reboques que chegam. Quando o número de trailers a serem removidos excede o número de portas de decapagem, os reboques que chegam são colocados em fila até que uma porta seja disponível. Um único reboque pode ter carga para vários ICPCs. Os reboques com frete destinado apenas ao ICPC local recebem uma prioridade mais baixa para acesso a uma porta de decapagem do que os trailers com frete de revezamento. De maneira semelhante, os reboques com frete de retransmissão que têm um destino final mais próximo têm prioridade mais baixa do que os reboques com frete de retransmissão com um destino final distante. O tempo para descarregar um contêiner e, se necessário, recarregar todas as suas cargas no um ou mais reboques de retransmissão é sempre de 2 horas, independentemente do tamanho e número de remessas. Um reboque de retransmissão é imediatamente encaminhado para seu próximo destino quando estiver cheio ou quando todas as remessas do dia esperados para esse destino tenham sido carregados no trailer. As remessas são medidas como uma porcentagem do volume de um reboque e podem ser subdividido até o percentual mais próximo para preencher o trailer. Há não há atraso entre a partida de um reboque e a chegada de outro reboque disponíveis nas portas de relé ou de decapagem. Nunca há escassez de trailers para distribuição posterior. Para ajudar a ACM a avaliar a eficiência de sua rede, você deve escrever um programa para determinar o tempo médio que um trailer espera por acesso a uma porta de decapagem e identificar as remessas que não serão cheguem integralmente em seus destinos intermediários ou finais no prazo. Entrada A entrada descreve um subconjunto possivelmente disjunto dos ICPCs da rede e padrões de tráfego que devem ser analisados. A primeira linha da entrada contém um número inteiro n que especifica o número de descrições de ICPC a serem processadas, 1 ≤ n ≤ 100. Isso é seguido de n descrições, cada uma descrevendo um ICPC. Cada descrição	Saída para a entrada de amostra: Não há espera para uma remoção porta no ICPC 0. O tempo médio de espera por uma remoção no ICPC 8 é de 63,3 minutos. As remessas atrasadas são: <table><tr><th>Id</th><th>Origem</th><th>Destino</th><th>Volume</th></tr><tr><td>17</td><td>11</td><td>8</td><td>40</td></tr><tr><td>23</td><td>11</td><td>10</td><td>15</td></tr><tr><td>33</td><td>3</td><td>10</td><td>35</td></tr><tr><td>19</td><td>18</td><td>10</td><td>50</td></tr></table>	Id	Origem	Destino	Volume	17	11	8	40	23	11	10	15	33	3	10	35	19	18	10	50
Id	Origem	Destino	Volume																		
17	11	8	40																		
23	11	10	15																		
33	3	10	35																		
19	18	10	50																		